

progress report

Tú Huỳnh Anh

March 2026

1 Uncertain Data

In traditional databases, the presence of an item in a transaction is considered deterministic, meaning an item either exists or does not exist. However, in many real-world applications, data may contain uncertainty due to noise, incomplete information, or measurement errors. Such data is referred to as **uncertain data**.

In an uncertain database, each item in a transaction is associated with a probability value that represents the likelihood of the item appearing in that transaction.

For example, a traditional transaction database may look as follows:

Transaction ID	Items
T1	A, B, C
T2	A, C
T3	B, D

In contrast, an uncertain transaction database represents items with probabilities:

Transaction ID	Items with Probability
T1	A(0.9), B(0.8), C(0.5)
T2	A(0.7), C(0.6)
T3	B(0.4), D(0.9)

The probability value indicates the likelihood that an item appears in a particular transaction. Uncertain data commonly appears in domains such as sensor networks, location tracking systems, and noisy data collection environments.

2 Frequent Itemset Mining

Frequent itemset mining is a fundamental task in data mining that aims to discover sets of items that frequently appear together in a transaction database. The frequency of an itemset is typically measured using a metric called **support**.

Support is defined as the number of transactions that contain a given itemset. An itemset is considered **frequent** if its support is greater than or equal to a user-defined minimum support threshold.

For example, consider the following transaction database:

Transaction ID	Items
T1	A, B, C
T2	A, C
T3	A, B
T4	B, C

If the minimum support threshold is set to 2, the frequent itemsets include:

- A (3)
- B (3)
- C (3)
- AB (2)
- AC (2)
- BC (2)

Frequent itemset mining is widely used in applications such as market basket analysis, recommendation systems, and pattern discovery.

3 Maximal Frequent Itemsets

A maximal frequent itemset is a frequent itemset that has no frequent supersets. In other words, it is a frequent itemset that cannot be extended by adding another item without violating the minimum support threshold.

For example, if the frequent itemsets are:

- A
- B
- C
- AB
- AC
- BC
- ABC

Then the maximal frequent itemset is:

- ABC

because no larger frequent itemset contains it.

Mining maximal frequent itemsets is beneficial because it significantly reduces the number of patterns that need to be stored and analyzed while still preserving essential information about the frequent patterns in the database.

4 Frequent Itemsets in Uncertain Databases

In uncertain databases, the traditional definition of support cannot be directly applied because items appear with certain probabilities. Therefore, the concept of **expected support** is used instead.

The expected support of an itemset is calculated by summing the probabilities that the itemset appears in each transaction.

For example, if a transaction contains items A(0.9) and B(0.8), the probability that the itemset {A, B} appears in the transaction is calculated as:

$$P(A, B) = 0.9 \times 0.8 = 0.72$$

The expected support of the itemset is then obtained by summing such probabilities across all transactions in the database.

5 Accumulated Dynamic Uncertain Databases

In many real-world scenarios, databases are not static but continuously evolve over time. New transactions may be added periodically, making the database dynamic.

A **dynamic database** is one in which the data changes over time due to insertions, deletions, or updates.

An **accumulated database** specifically refers to a database where new transactions are continuously appended while existing data remains unchanged.

For example:

Time	Database Content
t_1	T1, T2, T3
t_2	T1, T2, T3, T4, T5
t_3	T1, T2, T3, T4, T5, T6, T7

When uncertainty is also present, the resulting data structure becomes an **accumulated dynamic uncertain database**. Mining patterns from such databases is challenging because algorithms must efficiently update previously discovered patterns without reprocessing the entire database.

This motivates the development of algorithms capable of efficiently mining maximal frequent itemsets from accumulated dynamic uncertain databases.

6 The U-Apriori Algorithm

To mine frequent itemsets in uncertain databases, several algorithms have been proposed. One of the most well-known algorithms is the **U-Apriori algorithm**, which extends the classical Apriori algorithm to handle uncertainty in transaction data.

The classical Apriori algorithm relies on the *downward closure property*, which states that if an itemset is frequent, then all of its subsets must also be frequent. U-Apriori adopts a similar strategy but modifies the support calculation to work with probabilistic data.

Instead of counting the number of transactions containing an itemset, U-Apriori computes the **expected support** of an itemset. The expected support is defined as the sum of the probabilities that the itemset appears in each transaction.

Let X be an itemset and T_i be a transaction. If each item in the transaction is associated with a probability, the probability that the itemset X appears in transaction T_i can be computed by multiplying the probabilities of the items in X .

The expected support of itemset X is then defined as:

$$[\text{esup}(X) = \sum_{i=1}^n P(X, T_i)]$$

where $P(X, T_i)$ represents the probability that itemset X appears in transaction T_i , and n is the total number of transactions.

If the expected support of an itemset is greater than or equal to a user-defined minimum expected support threshold, the itemset is considered a frequent itemset in the uncertain database.

The U-Apriori algorithm follows a level-wise search strategy similar to Apriori:

1. Generate candidate 1-itemsets.
2. Compute the expected support of each candidate.
3. Prune itemsets whose expected support is below the minimum threshold.
4. Generate candidate k -itemsets from frequent $(k - 1)$ -itemsets.
5. Repeat the process until no more candidates can be generated.

Although U-Apriori is effective for mining frequent itemsets in uncertain databases, it assumes that the database is **static**. In many real-world applications, however, transaction databases are dynamic and continuously accumulate new data.

When new transactions are added to the database, U-Apriori must recompute the expected support of itemsets from the beginning, which can be computationally expensive for large datasets.

Therefore, more advanced approaches are required to efficiently mine frequent or maximal frequent itemsets in **accumulated dynamic uncertain databases**, where new transactions are continuously appended to the database.

7 Dataset Preparation

7.1 Data Pipeline

In this project, the Retail dataset was selected from the SPMF dataset repository as the benchmark dataset.

The Retail dataset is a transaction database where each transaction contains a list of purchased items. Each line represents a transaction and each number represents an item identifier.

Example of the original transaction database:

```
1 2 3 4
5 6
2 7 8
```

This means:

- Transaction 1 contains items $\{1,2,3,4\}$
- Transaction 2 contains items $\{5,6\}$
- Transaction 3 contains items $\{2,7,8\}$

Such datasets are widely used for frequent itemset mining and association rule mining.

7.2 Conversion to Uncertain Database

Since the focus of this project is mining frequent patterns from uncertain databases, the original transaction database was converted into an uncertain transaction database.

In an uncertain database, each item is associated with a probability value that represents the likelihood that the item actually appears in the transaction.

Example conversion:

Original transaction:

```
1 2 3
```

Converted uncertain transaction:

```
1:0.82 2:0.61 3:0.77
```

Another example:

```
5:0.93 6:0.54
```

```
2:0.71 7:0.68 8:0.59
```

In this representation:

- 1:0.82 means item 1 appears with probability 0.82
- 2:0.61 means item 2 appears with probability 0.61

To simulate uncertainty in real-world data collection, probability values were randomly generated between 0.5 and 1.0.

7.3 Dataset Scaling

To evaluate the scalability of the algorithm, the uncertain dataset was divided into several subsets with increasing sizes.

Examples include:

- Retail_uncertain_1000
- Retail_uncertain_5000
- Retail_uncertain_10000

These datasets contain:

Dataset	Number of Transactions
Retail_uncertain_1000	1000
Retail_uncertain_5000	5000
Retail_uncertain_10000	10000

Using datasets of different sizes allows the evaluation of runtime performance, memory usage, and algorithm scalability.

8 Mining Approach

The implemented method follows a U-Apriori style frequent itemset mining algorithm combined with maximal frequent itemset filtering.

8.1 Step 1 – Database Loading

The uncertain transaction database is loaded into memory. Each transaction is stored as a map structure:

```
Transaction = {item -> probability}
```

Example representation:

```
{  
  1 -> 0.82  
  2 -> 0.61  
  3 -> 0.77  
}
```

This structure allows efficient access to item probabilities during support computation.

8.2 Step 2 – Expected Support

Because the database is uncertain, traditional support cannot be used. Instead, expected support is used.

The expected support of an itemset X is defined as:

$$ExpSup(X) = \frac{1}{N} \sum_{t \in D} P(Xoccursint)$$

Where:

- N is the number of transactions
- D is the database
- $P(Xoccursint)$ is the probability that itemset X appears in transaction t

For example, for itemset $X = \{A, B\}$:

$$P(A, B) = P(A) \times P(B)$$

assuming item independence.

8.3 Step 3 – Candidate Generation

The algorithm follows the Apriori candidate generation strategy.

Frequent itemsets are generated level by level:

1-itemsets -> 2-itemsets -> 3-itemsets -> ...

The process continues until no new candidates can be generated.

8.4 Step 4 – Apriori Pruning

To reduce the search space, Apriori pruning is applied.

A candidate k -itemset is generated only if all of its $(k - 1)$ subsets are frequent.

This greatly reduces the number of candidates that need to be evaluated.

8.5 Step 5 – Maximal Frequent Itemsets

After all frequent itemsets are discovered, maximal frequent itemsets are extracted.

A frequent itemset X is maximal if no frequent itemset Y exists such that:

$$X \subset Y$$

Example:

Frequent itemsets:

{40}
{49}
{40,49}

Maximal frequent itemset:

{40,49}

because it has no frequent superset.

9 Results

frequent itemsets discovered from the dataset:

[33] -> 0.1189
[39] -> 0.1452
[40] -> 0.4253
[49] -> 0.3284

frequent 2-itemsets:

[39,40] -> 0.0716
[49,40] -> 0.1704
[40,42] -> 0.1091

frequent 3-itemsets:

[49,39,40] -> 0.0291
[49,40,42] -> 0.0500
[39,40,42] -> 0.0250

9.1 Maximal Frequent Itemsets

The maximal frequent itemsets obtained were:

[33, 42]
[39, 171]
[49, 39, 40]
[49, 40, 42]
[39, 40, 42]
[33, 49, 40]

These represent the largest frequent patterns discovered in the dataset.

9.2 Runtime Performance

Runtime: 203 ms